A Policy-Bounded Agentic Framework for Autonomous Governance of Cloud Data Pipelines

B Sai Krishna, Vignes VM, I Mahalakshmi, and Sri Harini MP

Abstract—A recent proposal governs cloud data pipelines with four policy-gated LLM agents and reports substantial gains over static orchestration. The original study does not disclose sufficient implementation detail to determine the exact model-in-the-loop evaluation condition behind those gains. We implement the system and evaluate it with a real, remote language model in a 560-run campaign that crosses four injected fault scenarios with eight configurations: three non-agent comparators, the full system, and four single-agent ablations. The runs are divided into live-agent, non-agent, and mock-model arms. With a live model, the full system changes mean time to recovery (MTTR) by −38% and cost by −56% relative to static orchestration, with most of the cost change coming from an assumed provisioning gap. Yet it increases manual interventions by 500%. Two idealized non-LLM baselines, each given a fixed modelled remediation latency, recover faster still, changing MTTR by −92% and −86%. When the same full configuration runs under a deterministic mock instead of the live model, recovery time falls to 0.193 s, about three orders of magnitude below the live figure, and the correct-mitigation score rises from 0.42 to 1.00. A mock evaluation and a live evaluation of the same system therefore support different conclusions. Graded against an oracle independent of the policy code, a generated adversarial corpus of 5547 cases yields containment ≥ 0.9990, but only after two contract-layer gaps exposed by the corpus were fixed. The evidence is bounded by a single host, synthetic data with injected faults, a simulated on-call engineer, one live model family, and baselines that are modelled rather than deployed.

Index Terms—LLM agents, cloud data pipeline governance, policy-as-code, Open Policy Agent, AIOps, evaluation methodology, adversarial evaluation, mock-vs-live evaluation validity

I. INTRODUCTION

Data pipelines fail in ordinary ways. An upstream schema changes a column's type, a source stalls, a co-tenant starves the pipeline of compute, or ingress bursts past capacity. An empirical study of real pipelines identifies 41 distinct factors behind such quality and reliability failures [1]. In most deployments the remedy is still an on-call engineer, and the time that engineer needs to respond often accounts for much of the recovery time. Large language model (LLM) agents could shorten this loop, and agent systems for cloud incidents already run in production [2], [3], [4]. Most of them stop at a recommended diagnosis. Once an agent may also act on production infrastructure, a governance question arises: what limits the actions it can take?

One recent answer [5] uses four bounded agents for monitoring, optimization, schema management, and recovery. Each agent emits only a structured action proposal, and a policy engine must approve the proposal before anything executes. Against static orchestration, the authors report reductions of up to 45% in recovery time, about 25% in cost, and more than 70% in manual interventions. Four gaps separate these claims from the evidence offered for them.

1) Evaluation path. The study names several model families as backends but gives too little implementation detail to identify the model-in-the-loop condition behind its headline figures. This matters because work on the sim-to-real gap finds that simulated evaluation environments can inflate agent performance [6], [7], [8].
2) Baselines. The only comparator is static orchestration, with fixed allocations, predefined retries, human-initiated remediation, and autoscaling disabled. Mature non-LLM automation, however, already runs at production scale [9].
3) Adversarial testing. The safety case rests on the policy gate, but a stated bound is not a tested one. Agent-security benchmarks report high attack success against weak defenses [10], [11], and the policy language itself has been criticized as too weak for agentic governance [12].
4) Statistics and generality. The study reports no confidence intervals, effect sizes, or run counts. It describes the GPT, Claude, and Gemini model families as interchangeable backends that differ only in verbosity and latency, yet presents no cross-model results, and other work finds that agent behavior varies across models and providers [13], [14].

We implement the system and ask five research questions.

RQ1: With a real model in the loop, does the full agent system outperform static orchestration on recovery time, cost, and manual interventions?
RQ2: Does it outperform cheap non-LLM automation, or only a slow human?
RQ3: Does it act correctly, and not merely quickly?
RQ4: Do the answers to RQ1–RQ3 change when the model is replaced by a deterministic mock?
RQ5: Does the policy gate contain adversarial proposals when judged against an oracle that is independent of the policy code?

RQ4 has the largest consequences. We ran the same full configuration under a deterministic mock and under a real, remote model. The two runs disagree by roughly three orders of magnitude on recovery time, and they

also disagree on whether the executed mitigation was correct (Section VI-E). The mock run is not wrong in any ordinary sense. It measures a faster and more forgiving experiment faithfully, but a governance claim drawn from that experiment does not transfer to the system an operator actually runs. The same caution applies to the other results in this paper: an evaluation that relies on the mock's near-zero latency, or on a cost model that assumes away the provisioning gap, reports gains that a live deployment does not deliver.

We count a contribution only if this campaign both implements and measures it. Four qualify.

1) A live-model evaluation of eight configurations (three non-agent comparators, the full system, and four single-agent ablations) across four injected fault scenarios. The campaign comprises 560 runs with per-run provenance and billing-accurate LLM accounting (Section V).
2) Explicit non-LLM baselines and a decision-quality metric. A rule-based responder and a threshold autoscaler, each modelled with a fixed remediation latency, run through the same campaign and show whether the agent system beats cheap automation as well as static orchestration. A correct-mitigation metric shows whether the system acts correctly, not only quickly (Section VI).
3) An adversarial evaluation against an independent oracle. A generated corpus is graded by a specification written separately from the policy code under test, so the containment rate does not rest on the policy grading itself (Sections VI and VII).
4) A disclosed cost model with closed-form sensitivity analyses for the two assumptions on which the headline numbers depend: the simulated human's latency and the provisioning gap (Eq. 4). We also report where the model's metered term falls short of a like-for-like measurement (Section VIII-D).

Two further components are implemented but were not exercised in this campaign, so we present them as architecture rather than as results. A cross-model harness could test the base paper's claim of model independence, but its live sweep has not been run against real providers. A production trust core with graduated autonomy, a durable kill switch, and a per-target blast-radius cap also exists, but every run used a single fixed mode, so there is no mode-transition or kill-switch latency to report. Section IV describes both, and Section VIII returns to them as concrete gaps for future work. Finally, Section VII records production-hardening lessons drawn from defects the system actually exhibited.

The architecture itself is not ours. Its three planes, four bounded agents, policy gate, and observe–reason–propose loop come from the base paper [5], which we implemented rather than redesigned. We claim no new agent design, policy language, or statistical method. Our contribution is the evidence, the measurement discipline that produced it, and the defects found by holding the system to its own stated guarantees instead of to a test suite that shares its assumptions.

The rest of the paper is organized as follows. Section II reviews related work, Section III formalizes the problem and the metrics, and Section IV describes the system as built. Section V presents the experimental design, and Section VI reports the results, centered on the mock-versus-live divergence. Section VII gives the hardening lessons, Section VIII discusses implications, limitations, and threats to validity, and Section IX concludes.

II. BACKGROUND AND RELATED WORK

This work draws on four bodies of literature: LLM agents for cloud operations, the validity of LLM agent evaluations, runtime governance of agent actions, and the non-LLM automation against which such agents should be measured. We review each in turn and then state how the present study differs.

A. The Base Architecture

The base paper [5] describes a data plane, a policy plane, and an agentic control plane. Four bounded agents share the control plane, and each produces only a validated action proposal. The evaluation compares the system with a static-orchestration baseline across injected failures. Our implementation follows the same decomposition. Wherever the description leaves a choice open, we record our decision and the alternatives in the accompanying artifact.

B. Heuristic Automation as the Baseline

Workflow orchestrators such as Airflow, Prefect, and Dagster are execution-centric. They schedule, retry, and alert, but they do not reason about failures their authors did not anticipate. Automation driven only by resource signals nonetheless achieves a great deal at production scale: Google's Autopilot [9], a heuristic autoscaler that covers a large share of Google's fleet, uses no LLM. Such automation is therefore a legitimate comparator rather than a strawman. An empirical study of real data pipelines catalogs 41 failure factors [1], and a comparison against static orchestration alone cannot show whether an agent system handles them better than cheap automation does. In our own evaluation, two idealized heuristic baselines outperform the full agent system on recovery time and manual interventions (Section VI).

C. LLM Agents for Cloud Operations

Reasoning-and-acting loops [15] are the general pattern behind agents that observe a system and propose actions. For root-cause analysis, RCACopilot [2] and RCAgent [3] apply tool-augmented LLMs to production incidents at Microsoft and Alibaba Cloud. Both stop at a recommended diagnosis. The design studied here also executes an action, and that step is what makes a policy gate necessary. OpsAgent [4], a multi-agent incident-management system in production at Lenovo, shows industrial practice moving in the same direction.

D. Evaluation Validity for LLM Agents

Our central finding is that a mock and a live evaluation of the same system diverge by orders of magnitude (Section VI-E). This finding belongs to a growing literature on the sim-to-real gap for LLM agents. The closest theoretical framing [6] decomposes the gap into observation, action, transition, and reward mismatches within a unified MDP view of foundation-model agents. Empirical studies point the same way. LLM-simulated users have been found to be "excessively cooperative" and to inflate agent success relative to grounded real-user data [7], and agent reliability drops when the same agents move from clean benchmark conditions to production-like stress such as task perturbation, tool faults, and rate limiting [8]. TheAgentCompany [16], a high-fidelity simulated workplace, finds that even its best agent completes only about 30% of consequential tasks, which identifies benchmark realism as a problem separate from the mock-versus-live question. Benchmark validity is also questioned more broadly: a survey of more than 1,300 agentic benchmark papers [17] finds that none meets basic procedural-validity criteria for public-sector deployment. Our contribution to this literature is a controlled mock-versus-live comparison of one system through one code path.

E. Graduated Autonomy and Runtime Policy

The trust core described in Section IV, with its shadow, approval, and autonomous modes, a durable kill switch, and a blast-radius cap, builds on the game-theoretic treatment of shutdown incentives [18]. A related framework [19] expresses the same idea in governance terms by separating an agent's permitted autonomy level from its capability level, and it demonstrates the separation on an enterprise data-engineering agent whose structure closely resembles the system studied here.

The policy gate runs on Open Policy Agent [20] and its Rego language. The base design's safety case is that the policy, not the model, bounds what can happen, and that case is only as strong as the policy's behavior on inputs its authors did not consider. One critique [12] argues that Rego-style languages are not expressive enough for agentic governance, and our adversarial corpus (Section VI) tests that concern empirically. ActGov [21] validates every proposed tool call against an SMT-verified policy engine before execution, which is functionally the same per-action gate that our ProposedAction contract and OPA policies implement.

F. Adversarial Evaluation

A policy bound that is asserted has not thereby been tested. Among extensible benchmarks, AgentDojo [10] and the Agent Security Bench [11], which reports attack success rates of up to 84.3% against weak defenses, are the closest to the corpus used here. The difference lies in the oracle. Ours is an independently written specification of the policy rather than the policy engine under test, so the containment figure does not depend on the policy grading itself.

G. Decision Quality and Cross-Model Generalization

Resolving an incident quickly is not the same as resolving it correctly. AIOpsLab [22] evaluates cloud agents on detection, localization, and mitigation rather than on latency alone, and OpenSec [23] finds that frontier models detect incidents well but are poorly calibrated, with false-positive containment-action rates of 82.5% and 45% for two current models. Our correct-mitigation metric (Section V) rests on the same distinction.

The base paper states that the GPT, Claude, and Gemini model families worked as interchangeable backends, but it reports no cross-model results. Evidence from other settings suggests caution. One study [13] measures a 43.4-point accuracy drop when a behavioral backdoor detector moves from in-distribution to cross-LLM evaluation across six production models. Another study [14] shows that, on identical input, agent behavior shifts with model version, open- versus closed-weight status, serving provider, and system prompt, and that weight-release status predicts behavior better than model identity. The cross-model harness described in Section IV was built to test this claim, but its live sweep was not run in this campaign (Section VIII).

H. Fault Injection and Statistics

Injecting faults deliberately to test resilience is established practice [24]. Our seeded injector reproduces four scenarios exactly: schema drift, upstream delay, resource contention, and ingress burst. For inference we use bootstrap resampling [25], the Mann–Whitney rank-sum test [26], the Wilcoxon signed-rank test [27], Holm's step-down correction [28], Cliff's δ as an ordinal effect size [29], and Wilson score intervals [30] for proportions near one, where the normal approximation collapses to a zero-width interval.

I. Positioning

The base paper presents policy-bounded agentic control as its advantage over unconstrained LLM-driven automation. However, it asserts that the bound holds without testing it against an adversary, offers no cross-model results for its claim of model independence, and compares only against static orchestration. We test the bound adversarially, compare against explicit, idealized non-LLM baselines, and state that the harness for the cross-model claim exists but has not been run live. Prior work on the sim-to-real gap already shows that agents can fail under live or adversarial conditions. This study adds evidence from one controlled campaign that the same system, run through the same code path, reaches qualitatively different conclusions depending on whether the model in the loop is mocked or live.

III. PROBLEM FORMULATION

A. Setting

A pipeline emits a telemetry snapshot s_k at each control tick k. The tick period τ is 15 s by default and was set

to 2 s in the evaluation campaign. A fault f is injected at time t_f, detected at $t_d \geq t_f$, and resolved at $t_r \geq t_d$, either by an executed mitigation or by an escalation that the on-call engineer handles. A configuration π is a responder that maps snapshots to actions. In an agent configuration, each enabled agent g proposes an action $a = (g, \text{type}, \text{target}, \text{params}, \text{justification}, \text{confidence})$ from the typed action space $\mathcal{A}$ of the contract (Section IV). A policy gate G decides each proposal from a context $c = (m, B, n, \kappa, v)$ that the system builds outside the agent's influence. Here m is the action's projected marginal cost, B the remaining budget, n the number of actions the proposing agent has recorded in the last ten minutes, κ the schema-compatibility class, and v a flag indicating whether a prior version exists. The gate returns allow, deny, escalate, or allow-and-notify.

B. Gate Semantics

The gate applies the rate limit of Eq. (1) before any agent-specific policy and fails toward the human:

$$G(a, c) = \text{deny} \quad \text{if } n \geq n_{\max}, \qquad n_{\max} = 5. \tag{1}$$

A proposal that matches no policy pack, or that cannot be evaluated because the policy engine is unreachable, is escalated rather than allowed. An optimization action that scales resources passes the cost policy only if the condition in Eq. (2) holds:

$$m \leq 0 \quad \text{or} \quad m \leq B. \tag{2}$$

A scale-down, whose marginal cost is non-positive, therefore always passes. Rollback is permitted only when v holds, and a schema change is permitted only when κ is backward-compatible. Section IV describes all four policy packs.

C. Metrics

Each run injects a single fault. Its recovery time, cost, and decision correctness are defined by Eqs. (3), (4), and (5):

$$\text{MTTR}(r) = t_r - t_d, \tag{3}$$

$$C(r) = C_{\text{meas}}(r) + \rho_c \, u_\pi \, T_h, \tag{4}$$

$$D(r) = \mathbf{1}\big[\text{executed mitigation} \in \mathcal{A}^*(f)\big]. \tag{5}$$

Here C_{meas} is the metered compute and storage cost, with compute priced at $\rho_c = 0.05$ per unit-second and storage at 0.01 per GB-hour; u_π is the allocation that configuration π holds over the horizon $T_h = 300$ s; and $\mathcal{A}^*(f)$ is the set of accepted mitigations for the injected fault. The three configurations that right-size their allocation, autoscale, optimization_only, and full, hold $u_\pi = 3$ units, and the others hold a static $u_\pi = 8$. Non-agent configurations never execute an agent mitigation, so $D(r)$ in Eq. (5) is 0 for them by construction.

Like the base paper, Eq. (3) measures recovery from detection. In the non-agent configurations a fixed monitor stamps detection at injection, $t_d = t_f$, so their MTTR also covers the detection interval. In an agent configuration the clock starts only when the monitoring agent flags the fault, which can be up to one tick plus one model call after injection. Manual interventions and data freshness are direct counts and lags read from telemetry.

Algorithm 1 One control tick.

```
record a heartbeat
if the kill switch is engaged then
    return
end if
run the monitoring agent: detect faults and stamp t_d
if an unresolved fault exists then
    P ← ∅
    for each enabled reactive agent g ∈ {schema, recovery, optimization} do
        a ← observe and reason (invalid output becomes no_action)
        add a to P unless it is no_action
    end for
    for each contested target, keep the proposal with the highest (agent priority, confidence)
    for each winning proposal a do
        skip if the target lock is held or the blast-radius cap is reached
        evaluate d ← G(a, c), then commit an intent row
        execute a if d allows it, or hand it to the on-call if d escalates
        update the audit row with the outcome
    end for
end if
```

D. The Evaluation-Validity Question

Let $\tilde{x}_{\text{live}}$ and $\tilde{x}_{\text{mock}}$ denote the median of metric x for the same configuration under a real language model and under a deterministic mock, respectively. We define the divergence ratio as

$$R_x = \frac{\tilde{x}_{\text{live}}}{\tilde{x}_{\text{mock}}}. \tag{6}$$

Reporting $\tilde{x}_{\text{mock}}$ as the system's performance is valid only when R_x in Eq. (6) is close to one. Section VI-E measures R_x for each metric, which answers RQ4 of Section I.

E. Threat Model

The adversary controls only its own proposal a. The context c, the policy engine, and the executor are trusted components built by the system. The safety property under test is containment: no proposal that the documented policy semantics classifies as unsafe may be allowed.

Algorithm 1 summarizes the control loop that Section IV describes in prose. Agent priority ranks recovery above schema and schema above optimization. The non-agent configurations skip the loop and resolve faults directly (Section V).

IV. SYSTEM

The system is organized in seven layers (Fig. 1). Layers 1 and 2 form the data plane and its telemetry, layer 3 holds the four bounded agents, layer 4 is the policy gate, and layer 5 executes and audits whatever the gate allows. Layers 6 and 7, orchestration and enterprise governance, extend the base paper's design. The campaign exercised layers 1 through 4, the write-ahead audit log in layer 5, and the control loop and arbitration in layer 6. It did not exercise the autonomy-mode transitions, did not engage the kill switch, and did not use layer 7. The remainder of this section describes the layers in order.

A. Layers 1 and 2: Data Plane and Telemetry

A batch pipeline in Apache Airflow ingests a TPC-DS-derived dataset into versioned warehouse partitions, and a streaming consumer reads from a Redpanda topic. All telemetry, including pipeline metrics, resource usage, task runs, agent actions, failure events, and manual interventions, is written to a PostgreSQL telemetry schema. Every experimental metric in this paper is reconstructed from that schema rather than logged separately. The same layer tracks data freshness and maintains the cost ledger behind the cost model of Eq. 4.

B. Layers 3 and 6: Agents and Orchestration

The four agents share one loop: observe a telemetry snapshot, reason with a live or mock language model, propose an action, and act through the gate. The monitoring agent is the detector and is enabled in every agent configuration, so recovery time remains measurable in every ablation. The optimization agent scales workers or pool slots, or reprioritizes work. The schema agent accepts compatible schema changes and contains breaking ones. The recovery agent retries, replays, rolls back, partially recomputes, or escalates to a human. When two proposals target the same resource, a static agent-priority bid resolves the conflict. An earlier design relied on lock ordering for this purpose, but lock ordering turned out not to provide the property, so a per-target advisory lock now serves only to stop a second process from acting on the same target.

C. The Contract Between Layers 3 and 4

An agent's only output is a ProposedAction with six fields: agent, action type, target, parameters, justification, and confidence. A typed schema validates each proposal and rejects any action type outside the agent's allowlist and any scaling target that is not an integer of at least one. The second check was added after the adversarial corpus (Section VI) showed that it was missing. Invalid output is rejected, logged, and counted as agent_output_invalid, and the agent's turn degrades to no_action. Agents never emit or execute code.

D. Layer 4: Policy Gate

Each proposal carries a context built by the system: the projected marginal cost, the remaining budget, the number of actions the proposing agent has taken in the last ten minutes, the schema compatibility, and whether a prior version exists. Four Rego policy packs evaluate the proposal: a rate limit (Eq. 1), a cost budget (Eq. 2), a recovery-approval policy that permits rollback only when a prior version exists, and a schema-compatibility policy. The resulting decision is allow, deny, escalate, or allow-and-notify. If the policy engine is unreachable, the gate escalates rather than allowing the action.

E. Layer 5: Execution and Audit

An intent row is written and committed before the executor runs, and it is updated with the outcome afterward. A crash during an action therefore leaves a durable executing row instead of no record at all. This write-ahead ordering was added later and was not part of the original design. An action dispatcher routes each allowed action type to a handler that performs its side effect: on Airflow, by triggering DAG runs, clearing tasks, and patching pool slots; on PostgreSQL, by rollback, quarantine, schema mapping, and desired-state updates.

F. Simulated Human

Escalations are resolved after a seeded log-normal delay (median 360 s, $\sigma = 0.5$) that stands in for the on-call engineer. This delay is a modeling choice rather than a measurement, and Section VI-G quantifies how strongly the headline results depend on it.

G. Cross-Model Harness, Implemented but Not Evaluated

A cross-model harness runs a scenario through a configurable set of LLM backends and scores decision correctness, latency, and token use per model, which directly tests the base paper's claim of model independence. The harness is unit-tested, but this campaign did not run or record the sweep across real provider models, so no result from it is reported here.

H. Layers 5 and 6: Trust Core, Implemented but Not Evaluated

Beyond the base paper's design, the system adds a graduated-autonomy layer with three modes. shadow mode logs proposals without touching the pipeline, approval mode queues allowed actions for a human decision, and autonomous mode executes them directly. A durable kill switch, checked once per control-loop tick, and a per-target hourly blast-radius cap bound the agents independently of the policy. All three mechanisms are implemented and unit-tested, but every configuration in the campaign ran in autonomous mode throughout, so this subsection describes a capability rather than a measured result.

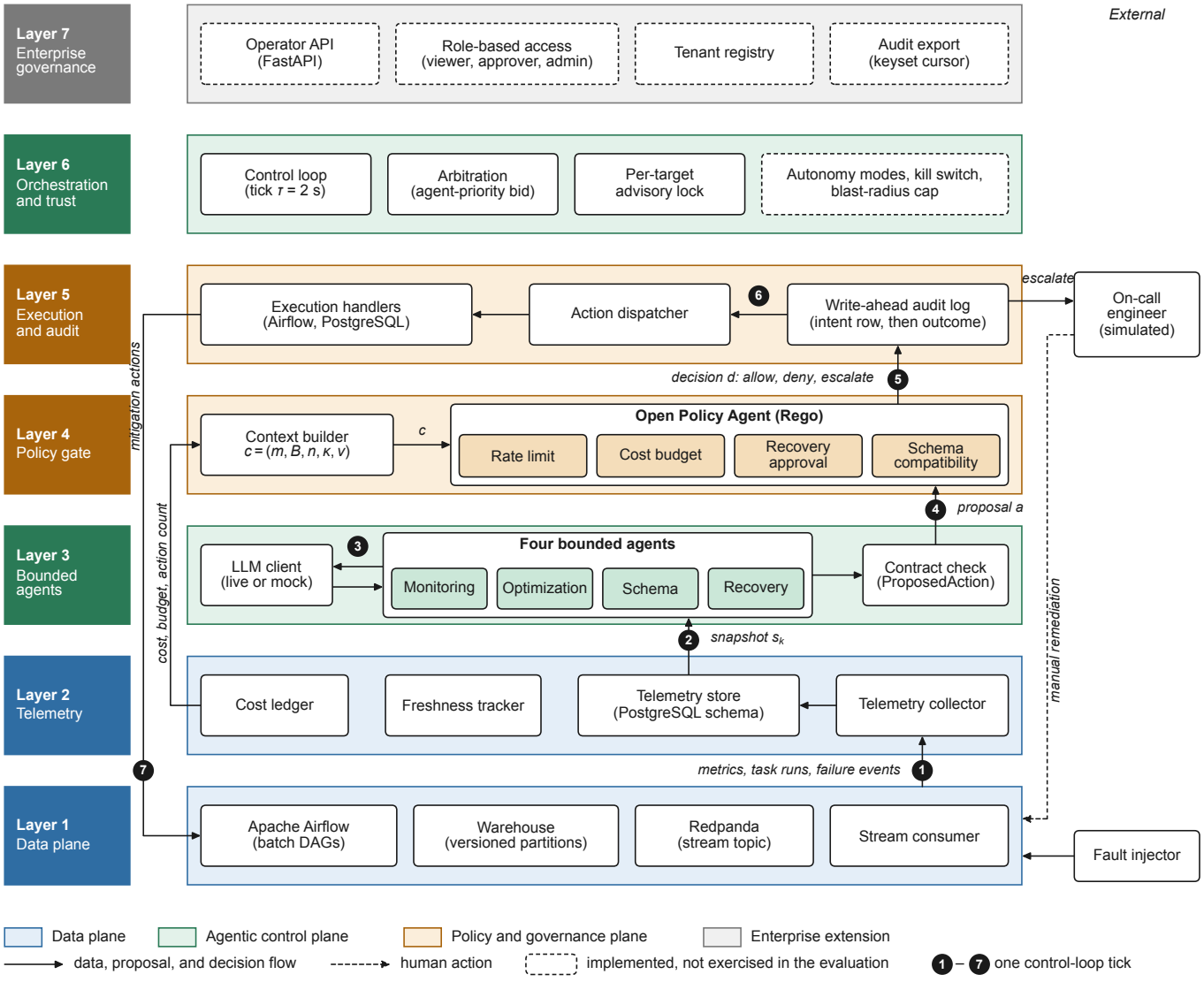


Fig. 1. Seven-layer system architecture. Tints group the layers into the three logical planes of the base design and the enterprise extension; numbered markers trace one control-loop tick.

I. Layer 7: Enterprise Governance, Implemented but Not Evaluated

An operator API provides read access to proposals, the audit trail, and costs, and it exports the full audit log through a keyset cursor. Role-based access control with viewer, approver, and admin roles, together with a tenant registry, binds each actor to a tenant and lets an operator suspend a tenant. This layer supports multi-tenant deployment but played no part in the experimental measurements.

V. EXPERIMENTAL DESIGN

A. Testbed

A single node runs the full stack in containers: PostgreSQL, Redpanda, Airflow, and OPA. Table I lists the hardware, software versions, and fixed parameters. The policy engine is pinned to a specific release, and Section VI shows why the pin matters. Runs execute strictly one at a time, because the resource-contention scenario applies real CPU stress and concurrent runs would add host load to the measurement.

B. Scenarios and Configurations

Four injected faults are crossed with eight configurations. The faults are schema drift (a column's type changes), upstream delay, resource contention, and an ingress burst. The configurations are baseline (static orchestration with the simulated human), rule_based (deterministic automation), autoscale (threshold autoscaling), full (all four agents), and four ablations. Three ablations add exactly one agent to monitoring: recovery_only, optimization_only, and schema_only. The fourth, monitor_only, runs monitoring alone. None of the three non-agent configurations calls a model.

TABLE I
Experimental Testbed and Fixed Design Parameters

Item	Setting
Host	One Apple M2 machine (arm64), 8 cores, 8 GB memory, macOS 15.7.2; runs are strictly sequential
Software	Docker 29.6.1; PostgreSQL 16.6; Redpanda v24.2.18; Apache Airflow 2.10.5 (Python 3.11); OPA 0.68.0
Batch data	Downscaled TPC-DS-derived synthetic fact table, 20,000 rows
Live model	Reasoning agents: nvidia/nemotron-3-ultra-550b-a55b; monitoring: nvidia/nemotron-3-super-120b-a12b; hosted OpenAI-compatible endpoint, temperature 0
LLM budget	60 calls and 150,000 tokens per run
Timing	120 s warm-up, 300 s control loop, 5 s settle; control tick 2 s (default 15 s)
Human model	Log-normal delay, median 360 s, $\sigma = 0.5$
Cost model	Static allocation 8 units, right-sized 3 units, horizon 300 s; 0.05 per compute unit-second, 0.01 per GB-hour
Baseline latencies	rule_based resolves covered faults 30 s after detection; autoscale 20 s after detection
Policy limits	Rate limit of 5 actions per 10 minutes; default cost budget of 100 units per run

C. Non-Agent Baselines

The three non-agent configurations model a responder's behavior rather than run a deployed controller. In baseline, every fault escalates to the simulated on-call engineer. In rule_based, a fault covered by the rule set (upstream delay, resource contention, or ingress burst) is resolved 30 s after detection, while schema drift, which requires reasoning, escalates to the human. In autoscale, the reactive scaler resolves resource contention and ingress bursts 20 s after detection, and every data fault escalates. The recovery times of these configurations therefore encode two assumed remediation latencies; they are not measurements of a real rule engine or autoscaler. They are idealized comparators: their speed follows from the assumed latencies, and any fault outside their coverage still falls to the human, which bounds what they can achieve.

D. Matrix and Arms

The baselines and full each have 80 runs pooled across the four scenarios, and the ablations have fewer; Table II gives the realized n for each configuration. Because only agent configurations call a model, the campaign is split into three independently resumable arms with separate result directories, so live and mock conditions cannot be pooled by accident. Arm A holds the live-model agent configurations. Arm B holds the non-agent configurations with the model disabled; their results are identical under live and mock by construction. Arm C runs full under a deterministic mock as the direct counterpart to arm A's full. The three arms contain 240, 240, and 80 runs, 560 in total. Every run begins with a 120 s warm-up before the fault is injected and ends with a 5 s settle. Agent runs execute a 300 s control loop in between, whereas the non-agent responders resolve the fault without a loop.

E. Live Model and Accounting

The live arm calls a remote language model at temperature zero through an OpenAI-compatible endpoint. The reasoning agents share one model, and monitoring uses a smaller, faster one. Model identifiers are recorded in the run manifest, and a per-run budget caps model calls and tokens.

Two properties of the accounting matter for validity. First, a proposal for a snapshot that has already been seen is served from an in-run cache, so summing per-action token columns would count one real call several times. We therefore count real calls, cache hits, degraded proposals, invalid outputs, tokens, and latency only at the moment a call is or is not actually made. A proposal counts as degraded when the budget is exhausted, the provider is unavailable, or an earlier degradation is replayed. Second, the project's pilot showed that caching a transient provider failure lets a single rate-limit episode degrade every later identical snapshot (Section VII). Transient failures are therefore no longer cached, and the degraded count is tracked separately.

F. Metrics

We report the five metrics defined in Section III: mean time to recovery (MTTR, Eq. 3), operational cost under a disclosed model (Eq. 4), manual interventions, data freshness, and decision correctness (Eq. 5). Manual interventions are a count of rows in a telemetry table that every configuration writes to, and freshness is a lag in seconds read from the same telemetry.

G. Seeds and Provenance

The seed of each run is sha256(config:scenario:replicate) mod 2^{32}. Because the configuration name is part of the hash, runs matched on (scenario, replicate) share no fault or human-latency randomness across configurations. The matching is therefore nominal rather than paired, so we treat the samples as independent and use the rank-sum test as the primary test; the signed-rank test on the matched index is kept only for continuity with earlier analysis. Each manifest line records the profile, the model mode and identifiers (never credentials), the timings, the

human-latency and provisioning parameters, the budgets, the code revision, and whether the experiment code had uncommitted changes. The analysis refuses to merge runs with differing provenance unless an explicit, documented override is given.

H. Statistics

We report medians with interquartile ranges, together with bootstrap 95% intervals for medians and for differences between medians. Each Cliff's δ is reported with its own bootstrap interval and read against the conventional thresholds: $|\delta| < 0.147$ negligible, 0.147–0.33 small, 0.33–0.474 medium, and ≥ 0.474 large [29]. Mann–Whitney U tests are Holm-corrected across the full family of comparisons, and proportions are given with Wilson intervals. All comparisons in Section VI are exploratory. The campaign was designed and run before any individual comparison was written up, and it was not pre-registered as a confirmatory study.

I. Adversarial Evaluation

A generated corpus probes the policy layer directly. It contains an exhaustive grid over every legal (agent, action) pair and the decision-relevant context values, including boundary cases such as 4, 5, and 6 recent actions and a cost equal to, or infinitesimally above, the budget. It also contains attacker-controlled parameter extremes priced by the real gate, hostile strings in free-text fields, illegal proposals sent directly to the policy engine without passing the contract, and a seeded fuzz. The expected verdicts come from an independently written Python statement of the documented policy semantics that shares no code with the Rego under test. The independence lies in the implementation, not in the authorship. Consistent with the threat model, the agent controls only its own proposal, and the surrounding policy context is built by the system and lies outside the agent's reach.

J. Sensitivity

Sampled human latency follows median $\cdot\, e^{\sigma Z}$, with Z fixed by the seed. In this scale family any all-human outcome scales exactly linearly with the assumed median. We confirm the linearity directly against the sampler and use it to derive, in closed form, the human median at which each configuration stops outperforming the baseline. For the cost model, we sweep the provisioning parameters and report where the break-even point lies.

VI. RESULTS

Every measured value in this section is generated directly from the campaign's raw telemetry (Section V) and can be traced to the script that produced it; only design constants and the base paper's reported figures are typed by hand. All 560 runs finished across the three arms, with no aborts and no token-ceiling breaches. The provenance check found 1 deviation across the merged arms: the campaign supervisor was restarted mid-campaign after a host infrastructure failure unrelated to the experiment. The fix deployed with that restart changed only the supervisor's own health check and none of the code on a run's execution path, which we confirmed by diff before treating the arms as one dataset. The first three subsections answer RQ1–RQ3 with the live model, and Section VI-E then shows that these answers depend on whether the model was real (RQ4).

A. Does the Full Agent System Beat Static Orchestration?

Table II reports medians and interquartile ranges per configuration, and Table III compares each configuration with baseline using bootstrap confidence intervals, Cliff's δ, and Holm-corrected Mann–Whitney p-values. Figures 2 and 3 show the underlying distributions and effect sizes. The full configuration changes MTTR from 378 s to 235 s ($\Delta = -143$ s, -38%, Cliff's $\delta = -0.56$, $p_{\text{Holm}} < 0.001$) and cost from 158 to 69.8 units (-56%, $\delta = -0.66$, $p_{\text{Holm}} < 0.001$). Both effects are significant, but the MTTR reduction is far smaller than the one the mock evaluation of the same configuration produces (Section VI-E).

Cost is the only headline metric whose change reaches the base paper's reported 25% reduction, but the reduction depends on the assumed provisioning gap (Section VI-G). Most of it comes from the provisioning term of Eq. (4), which is an assumption, rather than from metered compute. The pooled figure also hides two features of the cost data. First, the saving depends on the scenario: the median cost of full is below that of baseline in three scenarios but above it in ingress_burst (523 against 158 units). Second, agent configurations that do not right-size cost far more than baseline in the pooled data: monitor_only and recovery_only by 676% and 676%, and schema_only by 484%. Because these configurations hold the same static allocation as baseline, the excess lies entirely in the metered compute term, and it is largely an artifact of how that term is measured (Section VIII-D). Resources are sampled once before injection and once after the settle, so an agent configuration meters compute over its full 300 s loop, whereas a non-agent configuration, whose response completes without waiting, meters only a few seconds. The saving is therefore better attributed to the right-sizing configurations, chiefly the optimization agent, than to agent control in general.

Manual interventions move in the opposite direction. The median rises from 1.00 to 6.00 ($\Delta = 5.00$, 500%, $\delta = 0.79$, $p_{\text{Holm}} < 0.001$), the reverse of the base paper's reported reduction of more than 70%, and recovery_only shows the same rise (500%). One plausible mechanism is that baseline escalates exactly once per injected fault by construction, whereas an agent configuration that is rate-limited or denied by the budget can propose again on a later tick of the same 300 s loop, with each cycle adding an escalation. We did not instrument which mechanism dominates, so we report the measurement and leave the mechanism open (Section VIII). Because reducing manual

TABLE II
Medians [IQR] per Configuration, With n Replicates Pooled Across Scenarios

Config	LLM	n	MTTR (s)	Cost	Manual interv.	Decision correct
baseline	none	80	378 [244]	158 [113]	1.00 [0]	0 [0]
rule_based	none	80	30.0 [15.9]	158 [113]	0 [0.250]	0 [0]
autoscale	none	80	52.4 [359]	82.5 [122]	0.500 [1.00]	0 [0]
monitor_only	live	40	359 [210]	1,223 [525]	1.00 [1.00]	0 [0]
recovery_only	live	40	220 [75.0]	1,223 [630]	6.00 [1.00]	0 [0]
optimization_only	live	40	267 [360]	76.5 [128]	1.00 [1.00]	0 [0.250]
schema_only	live	40	315 [353]	920 [122]	1.00 [1.25]	0 [0.250]
full	live	80	235 [143]	69.8 [35.1]	6.00 [8.00]	0 [1.00]

TABLE III
Effect of Each Configuration Against baseline: Bootstrap 95% CI on the Median Difference, Cliff's δ With Its Own 95% CI, and Holm-Corrected Mann–Whitney p-Value ($^*p_{\text{Holm}} < 0.05$)

Config	Metric	Δ median [95% CI]	Cliff's δ [95% CI]	p_{Holm}
rule_based	MTTR (s)	−348 [−419, −303]	−0.73 [−0.85, −0.60]	<0.001*
autoscale	MTTR (s)	−326 [−406, −82.7]	−0.49 [−0.63, −0.32]	<0.001*
monitor_only	MTTR (s)	−19.2 [−99.6, 59.8]	−0.09 [−0.32, 0.13]	1.000
recovery_only	MTTR (s)	−159 [−229, −105]	−0.70 [−0.83, −0.55]	<0.001*
optimization_only	MTTR (s)	−111 [−188, 15.9]	−0.27 [−0.50, −0.025]	0.134
schema_only	MTTR (s)	−63.1 [−182, 87.4]	−0.19 [−0.42, 0.056]	0.638
full	MTTR (s)	−143 [−212, −94.7]	−0.56 [−0.70, −0.42]	<0.001*
rule_based	Cost (units)	0 [0, 0]	0.00 [−0.14, 0.14]	1.000
autoscale	Cost (units)	−75.0 [−75.0, −75.0]	−0.62 [−0.76, −0.45]	<0.001*
monitor_only	Cost (units)	1,065 [1,065, 1,074]	1.00 [1.0, 1.0]	<0.001*
recovery_only	Cost (units)	1,065 [1,065, 1,389]	1.00 [1.0, 1.0]	<0.001*
optimization_only	Cost (units)	−81.0 [−81.0, −48.3]	−0.47 [−0.72, −0.20]	<0.001*
schema_only	Cost (units)	763 [750, 763]	1.00 [1.0, 1.0]	<0.001*
full	Cost (units)	−87.7 [−96.2, −87.7]	−0.66 [−0.81, −0.51]	<0.001*
rule_based	Manual interv.	−1.00 [−1.00, −1.00]	−0.75 [−0.85, −0.65]	<0.001*
autoscale	Manual interv.	−0.500 [−1.00, 0]	−0.50 [−0.61, −0.39]	<0.001*
monitor_only	Manual interv.	0 [0, 0]	0.28 [0.15, 0.42]	<0.001*
recovery_only	Manual interv.	5.00 [5.00, 5.50]	0.97 [0.93, 1.0]	<0.001*
optimization_only	Manual interv.	0 [0, 1.00]	0.20 [−0.050, 0.42]	0.121
schema_only	Manual interv.	0 [0, 0]	0.25 [0.12, 0.38]	<0.001*
full	Manual interv.	5.00 [5.00, 8.00]	0.79 [0.70, 0.88]	<0.001*
rule_based	Decision correct	0 [0, 0]	0.00 [0, 0]	1.000
autoscale	Decision correct	0 [0, 0]	0.00 [0, 0]	1.000
monitor_only	Decision correct	0 [0, 0]	0.00 [0, 0]	1.000
recovery_only	Decision correct	0 [0, 0]	0.00 [0, 0]	1.000
optimization_only	Decision correct	0 [0, 0]	0.25 [0.12, 0.38]	<0.001*
schema_only	Decision correct	0 [0, 0]	0.25 [0.12, 0.38]	<0.001*
full	Decision correct	0 [0, 1.00]	0.42 [0.31, 0.54]	<0.001*

interventions is the system's stated purpose, we count this increase as a negative result for the design. The answer to RQ1 is therefore mixed: with a live model, the full system beats static orchestration on recovery time and cost, but not on manual interventions.

B. Is the Full System Better Than Cheap, Non-Agent Automation?

The rule_based and autoscale configurations call no model, yet both beat every agent configuration on MTTR, including full. This ranking depends on the modelled remediation latencies of Section V. Both comparators resolve covered faults after a fixed 30 s or 20 s, so they show what an idealized non-LLM responder could achieve rather than the measured speed of a deployed one. rule_based reaches 30.0 s (−92% relative to baseline, $\delta = -0.73$), and autoscale reaches 52.4 s (−86%, $\delta = -0.49$), compared with −38% for full. Both keep manual interventions flat or reduce them (−100% and −50%), whereas full increases them.

On cost, autoscale changes the median by −48%, a smaller reduction than the −56% of full. rule_based, which does not right-size, shows no cost effect (0%, $p_{\text{Holm}} = 1.000$), as its design predicts. By construction, neither comparator can execute an accepted mitigation (decision_correct= 0), and decision quality is the one axis on which full wins outright (Section VI-C). For these fault scenarios and this control-loop timing, cheap deterministic automation beats the LLM-driven system on two of the four metrics. For idealized comparators, then, the answer to RQ2 is negative: the agent system does not beat cheap automation on speed or on manual interventions.

C. Decision Quality

The decision_correct metric is scored against a per-scenario set of accepted mitigations (Eq. 5, Section V), and every non-agent configuration scores zero by construction. Among the agent configurations, only full ($\delta = 0.42$, $p_{\text{Holm}} < 0.001$), optimization_only ($\delta = 0.25$), and schema_only ($\delta = 0.25$) change the metric significantly;

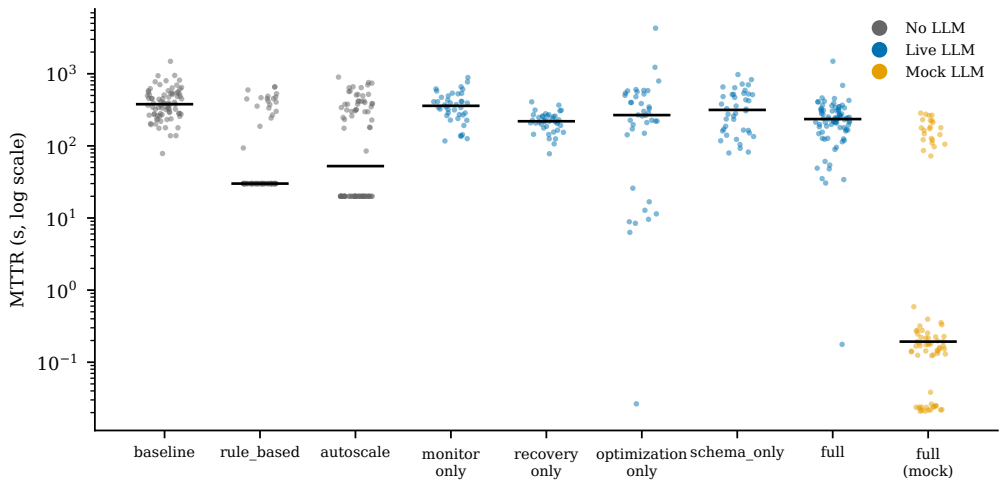


Fig. 2. Recovery-time distribution by configuration (log scale).

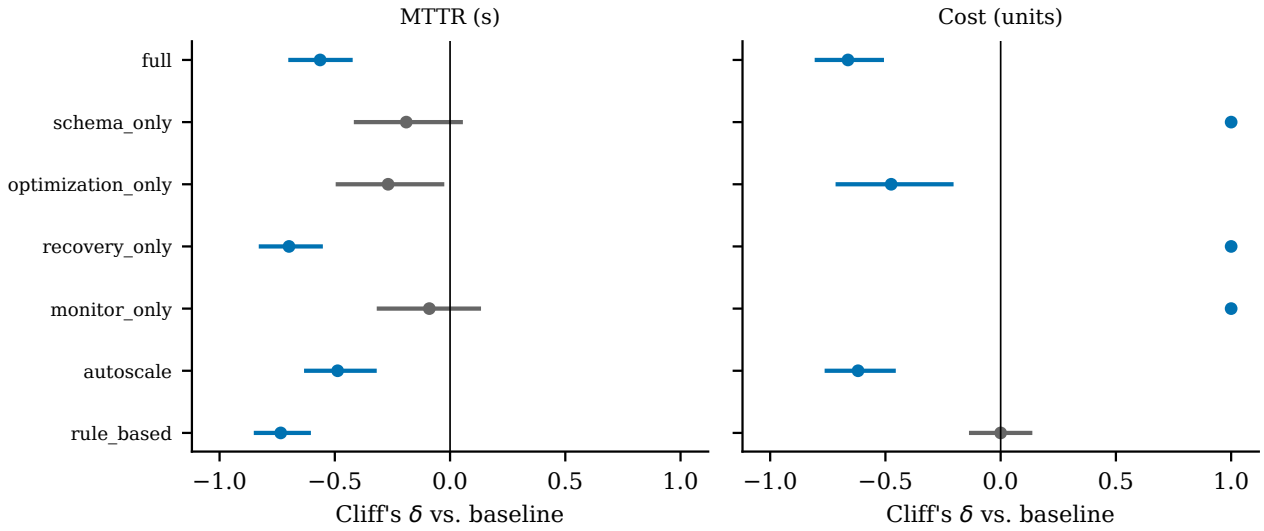


Fig. 3. Cliff's δ against baseline, by configuration and metric, with bootstrap 95% intervals.

monitor_only and recovery_only do not. Against an all-zero comparator, Cliff's δ equals the treatment group's success rate, so these values are also correct-mitigation rates: full executes an accepted mitigation in 0.42 of its runs. This is the one metric on which the live system achieves something that neither the static baseline nor the rule-based comparator can achieve at all. It answers RQ3 only in part, however, because fewer than half of the full system's runs act correctly. As Section VI-E shows, the mock evaluation scores the same configuration at 1.00.

D. Per-Scenario Heterogeneity

Table IV and Fig. 4 break MTTR down by configuration and scenario. No single configuration dominates every fault type. optimization_only is the fastest configuration on ingress_burst but is slower than baseline on schema_drift, a fault on which its only non-monitoring agent has no lever, and schema_only shows the mirror-image pattern. This is the pattern a single-agent ablation should produce: the agent helps where it owns the fault and is inert, or pure overhead, elsewhere. It also cautions against treating any single scenario's result as a configuration's typical behavior.

E. Mock Versus Live: The Central Finding

Table V and Fig. 5 compare full under two conditions that differ only in the model: arm A (live) and arm C, its deterministic-mock counterpart. MTTR falls from 235 s live to 0.193 s mock, a divergence ratio of $R_{\mathrm{MTTR}} = 1219$ (Eq. 6). The median decision-correct score rises from 0 to 1.00, although the live correct-mitigation rate is 0.42. Manual interventions fall from 6.00 to 0, and freshness falls from 14.0 s to 0.0801 s.

TABLE IV
Median MTTR (s) by Configuration and Scenario

Config	ingress_burst	resource_contention	schema_drift	upstream_delay
baseline	352	386	424	353
rule_based	30.0	30.0	433	30.0
autoscale	20.0	20.0	363	393
monitor_only	366	412	320	275
recovery_only	223	183	224	248
optimization_only	10.5	267	502	281
schema_only	453	343	120	553
full	148	318	214	237

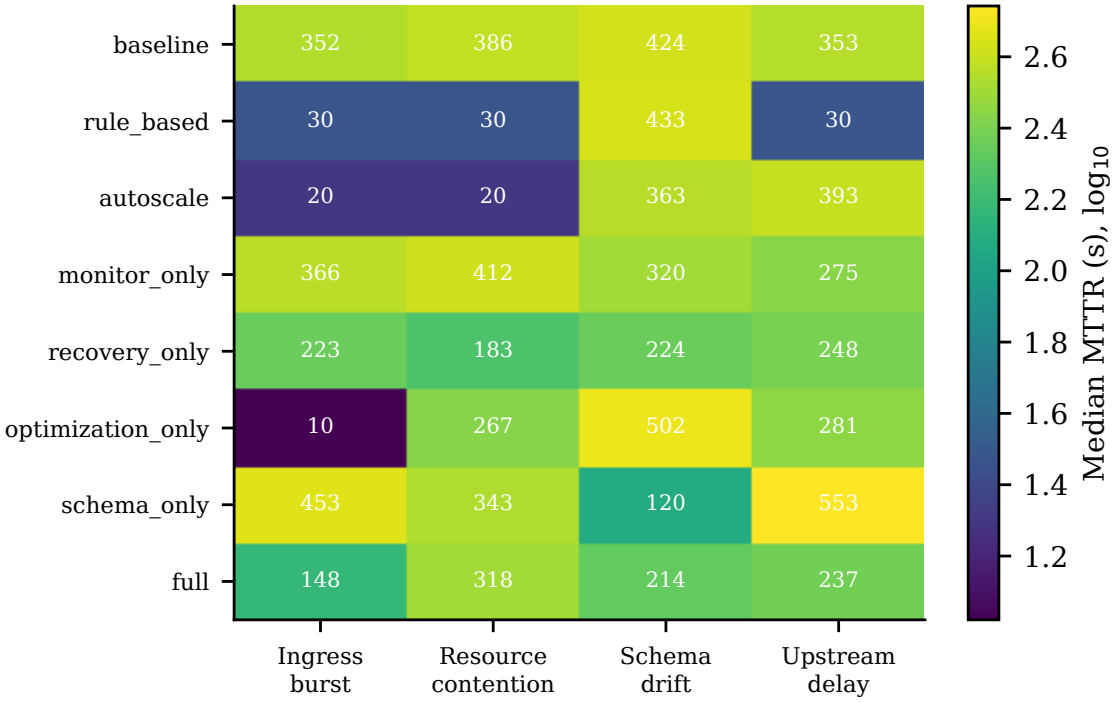


Fig. 4. MTTR by configuration and scenario.

Cost moves the other way: the live arm is cheaper than the mock arm (69.8 vs. 833 units), and the mock's cost even exceeds that of baseline. Much of this gap likely reflects the harness rather than the model. The Airflow pool size and the streaming-worker target are global settings that the campaign did not reset between runs, so each run inherits the state left by the previous agent action. The mock arm ran last, after schema_only. In the schema-drift and upstream-delay scenarios, where the mock's rule-based optimization agent takes no action, mock full meters exactly the same compute as live schema_only. The live full arm ran first, from a different starting state, and its metered compute stayed low in three of the four scenarios. The cost comparison between the two arms is therefore not a clean measurement of the model's effect, and we do not interpret it as one.

These results answer RQ4. The mock evaluation is not a noisier or scaled-down version of the live one. On recovery time and decision quality it is a categorically different and far more flattering experiment, and the earlier results in this section should be read with that in mind. The base paper's headline figures came from an evaluation path that its description does not identify, and for one system this campaign measures how far the evaluation path alone can move the answer. A claim about this class of system drawn from a mock run is a claim about the mock, not about the model it replaces.

F. LLM Accounting

Table VI reports real model calls, cache hits, degraded proposals, invalid outputs, tokens, and latency for each agent configuration, counted at the moment a call is or is not made (Section V). Degraded and invalid rates are zero across the whole campaign: no run exhausted its budget or met a provider outage, and no schema-invalid output reached the final runs. This is a property of this campaign,

TABLE V
Live Model Versus Deterministic Mock for full Under Otherwise Identical Configuration and Scenarios, With Cliff's δ (95% CI) and Holm-Corrected p

Metric	Live	Mock	Cliff's δ [95% CI]	p_{Holm}
MTTR (s)	235	0.193	0.81 [0.72, 0.90]	<0.001
Cost (units)	69.8	833	−0.82 [−0.92, −0.71]	<0.001
Manual interv.	6.00	0	0.87 [0.79, 0.93]	<0.001
Decision correct	0	1.00	−0.57 [−0.69, −0.46]	<0.001
Freshness (s)	14.0	0.0801	0.25 [0.074, 0.42]	0.004

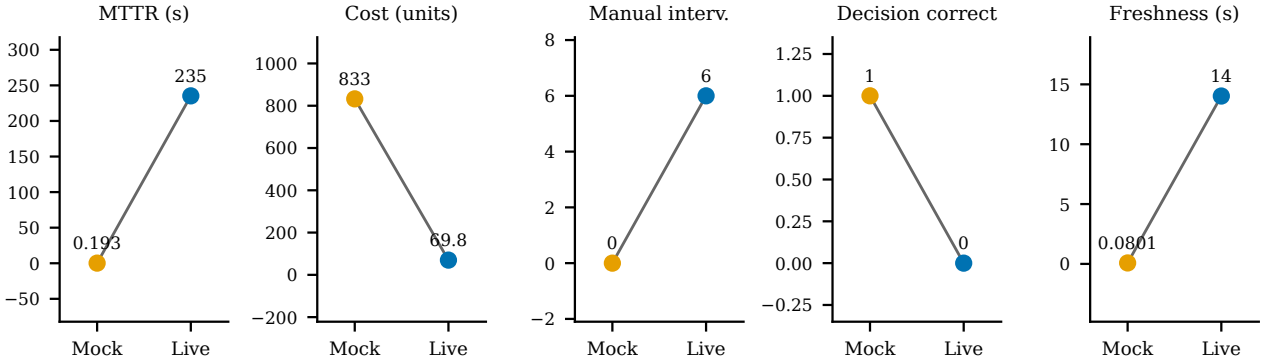


Fig. 5. Live versus mock medians for full, per metric. The axes are linear because two metrics are exactly zero for one arm, which a log axis cannot show.

TABLE VI
LLM Accounting per Agent Configuration, Counted When a Call Is or Is Not Made

Config	Calls	Cache hits	Degraded	Invalid	API tokens	Latency (s)	Degraded %
monitor_only	1.00	143	0	0	1,244	4.37	0%
recovery_only	2.00	262	0	0	2,737	10.5	0%
optimization_only	2.00	258	0	0	2,998	9.22	0%
schema_only	2.00	274	0	0	2,550	5.73	0%
full	4.00	214	0	0	5,946	15.0	0%

not a standing guarantee. The pilot that preceded it found a live cache-poisoning defect, exactly the kind of failure the accounting is designed to catch, and the defect was fixed before the campaign began.

The agents' overhead is small in absolute terms. Across the 240 live-arm runs, the agents made 703 real model calls and consumed 975,570 API tokens in total, and no run came close to the per-run budget: the largest used 16,042 tokens against a cap of 150,000, and at most 11 calls against a cap of 60. The per-run model latency in Table VI is small compared with the 300 s control loop, so inference time alone does not explain the live system's recovery time. We do not convert tokens into a monetary cost, because the provider's per-token prices were not recorded with the runs; token counts are the portable measure.

G. Sensitivity

Table VII and Figs. 6 and 7 present the two closed-form sensitivity analyses of Section V. full continues to beat the simulated human down to a median human latency of 224 s, well below the assumed 360 s but far above the 28.5 s break-even of rule_based. The rule-based responder's speed advantage therefore survives a much faster on-call engineer than that of full. On cost, the reduction of full holds down to a static provisioning level of 2.15 units; below that level, a baseline that was never heavily over-provisioned removes the saving on its own. Both analyses make the point of Section VI-E from another angle: the headline numbers depend on modeling choices, so we report each number together with the condition it depends on.

H. Adversarial Containment

Table VIII grades the generated corpus (Section V) against the independent specification oracle, by category and overall, and answers RQ5. Overall containment is 3867/3867 ($\geq$ 0.9990, Wilson 95%). The over-block rate on legal proposals is zero, with the exact denominator given in the table. The policy agrees with the oracle on all 5547/5547 graded cases, with 0 gate errors. These figures were measured after the two contract-layer fixes described in lesson L6 (Section VII), which an earlier pass of the

TABLE VII
Closed-Form Break-Even Points: Assumed Human-Latency Median (s) and Static Provisioning (Units) at Which Each Configuration Stops Beating Its Comparator

Config	MTTR (s)	Human break-even (s)	Cost reduction (%)	Static break-even
autoscale	52.4	49.9	47.6	3.00
full	235	224	55.7	2.15
monitor_only	359	342	−676	–
optimization_only	267	254	51.4	2.60
recovery_only	220	209	−676	–
rule_based	30.0	28.5	0	–
schema_only	315	300	−484	–

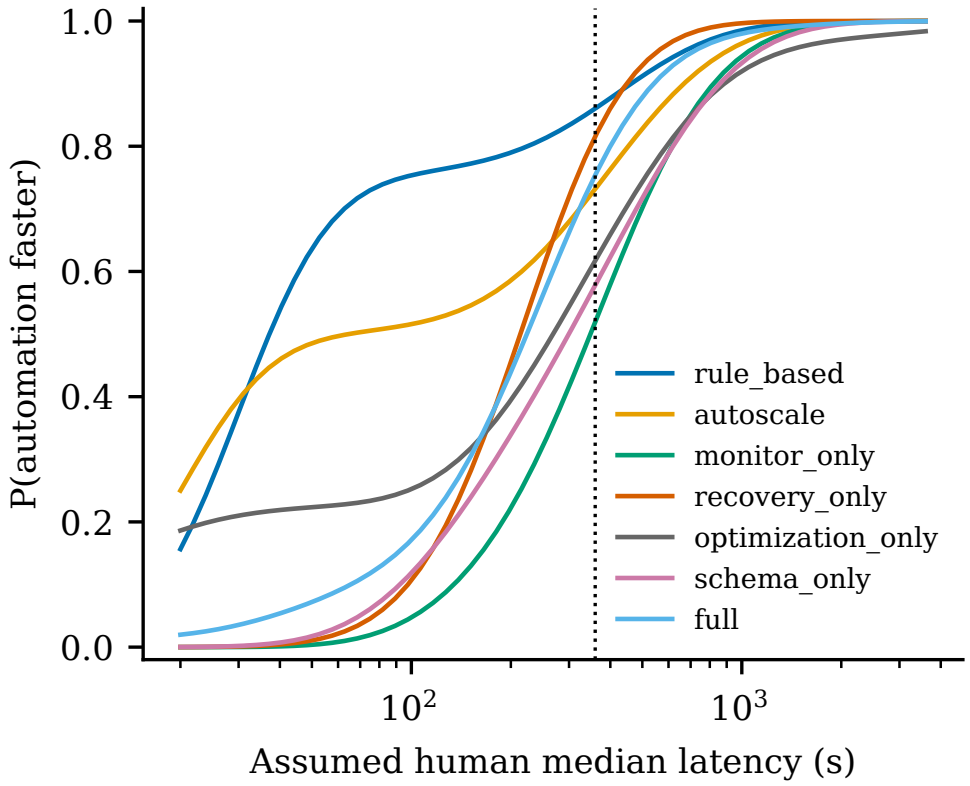


Fig. 6. Probability that automation recovers faster than the simulated human, as a function of the assumed human median latency.

same corpus made necessary. They therefore describe the fixed system, and the corpus that exposed the gaps is now a committed regression test for this claim.

VII. PRODUCTION-HARDENING LESSONS

The central finding of Section VI-E has a less statistical counterpart. The system had several defects that its own test suite could not see, for the same reason that the mock evaluation flatters the headline numbers: the path the tests exercised was not the path production takes. Each lesson below was found through an audit or an experiment rather than through a failing test, and each is documented in the accompanying artifact.

A. L1: The Shared Resource Nobody Declared Concurrency-Unsafe

The base design states that execution is deterministic and auditable. Versioned partition creation, however, read the current maximum version and then dropped, created, and registered a table in separate statements without a lock, so two concurrent callers targeting the same partition could compute the same next version and race. The defect appeared as an intermittent continuous-integration failure, and we traced it to its root instead of retrying it away: a recovery-agent replay had triggered the same DAG as an unrelated test. The fix serializes the read-modify-write behind a database advisory lock. Lesson: once an agent can trigger real work, every shared resource it touches becomes a concurrency surface that a single-actor test

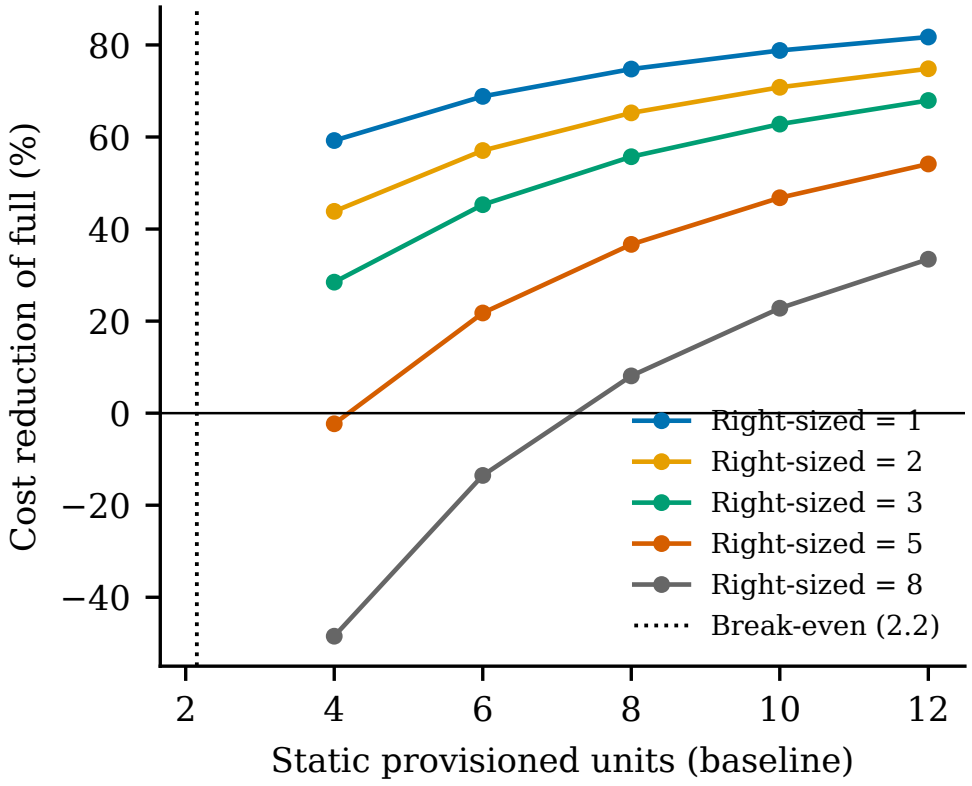


Fig. 7. Cost reduction of full as a function of the assumed static provisioning level.

TABLE VIII
Adversarial Corpus Graded Against the Independent Specification Oracle, by Category

Category	Cases	Containment [Wilson 95%]	Over-block	Oracle agreement	Fail-open
boundary_grid	4320	3056/3056 [0.9987, 1.0000]	0/1264 [0.0000, 0.0030]	4320/4320 [0.9991, 1.0000]	0
defense_in_depth	133	133/133 [0.9719, 1.0000]	–	133/133 [0.9719, 1.0000]	0
fuzz	1000	606/606 [0.9937, 1.0000]	0/394 [0.0000, 0.0097]	1000/1000 [0.9962, 1.0000]	0
injection	28	28/28 [0.8794, 1.0000]	–	28/28 [0.8794, 1.0000]	0
param_extremes	66	44/44 [0.9197, 1.0000]	0/22 [0.0000, 0.1487]	66/66 [0.9450, 1.0000]	0
Overall	5547	3867/3867 [0.9990, 1.0000]	0/1680 [0.0000, 0.0023]	5547/5547 [0.9993, 1.0000]	0

never exercises.

B. L2: An Action Could Be Lost From the Audit Trail

The audit row was originally written after execution. A crash, an out-of-memory kill, or a database outage in that window could therefore leave an action that genuinely happened with no record, and because the control loop suppresses per-tick exceptions, the failure was silent. For a system whose central claim is that every action is policy-gated and auditable, an executed action without an audit record contradicts that claim directly. The intent row, which already carries the policy verdict, is now committed before execution and updated with the outcome afterward. Lesson: record the intent before the action, not after it.

C. L3: The Detector That Had No Callers

The table on which both the recovery-time statistics and decision scoring depend was written only by the fault injector. The monitoring agent updated the detection time of an existing row but never created a new row, which left a complete, unit-tested, deterministic anomaly detector with no caller. In a deployment without the injector, a genuinely detected incident would have reached the audit table but never become an incident record, so production could not have reproduced the experiment's own metrics. We found the gap while planning a later feature, by reading the code rather than through a failing test. Lesson: a component can pass every test it owns and still be disconnected, so trace the data path from production input to the reported metric.

D. L4: Two Taxonomies That Never Met

Decision correctness is scored against an accepted-mitigation set (Eq. 5) keyed by the name of the injected scenario, whereas live incidents carry different fault-type names. Applying the set to live incidents would have

marked every real incident "incorrect by construction." A mapping from live fault types onto the same accepted-mitigation set was added before the metric was exposed outside the experiment harness. Lesson: a metric defined in the experiment's own vocabulary cannot move into production without an explicit mapping.

E. L5: Hermeticity by Enumeration Does Not Scale

One class of unit-test failure kept recurring: a test that did not mock one module's database handle turned a millisecond test into a thirty-second connection-pool timeout. We fixed it instance by instance four times before addressing the mechanism. Every such call now goes through a single connection-pool accessor that is patched to fail fast under test. Mutation testing of this guard, by removing one essential mock, made six tests fail within seconds, each naming the exact cause. Lesson: when the same bug appears a fourth time, fix the mechanism rather than the instance.

F. L6: What the Adversarial Corpus Found

Grading a generated corpus against an independent oracle revealed three things (numbers in Section VI). First, the contract layer had two gaps. The cost policy priced a scaling target against the budget but did not check its floor. Zero workers, which halts ingestion entirely, was therefore within budget, and so was a negative "scale-down." In addition, non-numeric targets raised an exception inside the gate before the audit row was written, so a rejected proposal could leave no record. Both gaps are now closed in the contract layer: scaling targets must be integers of at least one, and any violation follows the ordinary invalid-output path.

Second, under a newer policy-engine release than the one this project pins, some floating-point budget comparisons evaluated as within budget when they were not, whereas the pinned release behaved correctly. We report this observation without having traced its root cause or reported it upstream, because the consequence holds either way: the version pin is essential, and the corpus now serves as an integration test that would catch a version upgrade that changes these semantics again.

Third, once the contract fixes were in place, the policy agreed with the independent specification on every graded case. The containment claim is therefore a measured property with a confidence interval rather than an assertion repeated from the code under test. Lesson: a safety property asserted by a policy's own authors is not evidence until an independent oracle has tried to break it.

G. L7: The Pilot Found a Retired Model

Before the live campaign, an eight-run pilot at the real paper timings measured cost and wall time. Within minutes it found that the provider had retired the small model configured for the monitoring agent (HTTP 410, end of life). Because monitoring is the detector in every agent configuration, the full campaign would otherwise have run with a detector degraded to a no-op, and every downstream number would have measured the provider's retirement of the model rather than the system. The supervisor's degraded-streak guard would eventually have caught the problem, but only after several wasted runs; the pilot caught it on the first call. Lesson: the cheapest validity check in a long, paid experiment is a short run at the real settings, because a model that a provider lists is not necessarily a model that still answers.

VIII. DISCUSSION

A. Implications for Evaluating Governed Agents

The mock-versus-live divergence (Section VI-E) bears directly on how systems of this kind should be evaluated, and four practices follow from the results and from the lessons of Section VII. First, state whether the model in the loop was real or mocked, and report both numbers whenever both exist, because a mock figure describes only the mock. Second, include cheap non-LLM baselines: here two idealized baselines beat the full agent system on recovery time and manual interventions, which a comparison against static orchestration alone would have concealed. Third, grade any safety claim against an oracle that shares no code with the mechanism under test; the independent oracle used here found two contract-layer gaps that the system's own tests had missed. Fourth, run a short, paid pilot at the real settings before committing to a long campaign; ours detected a retired model on its first call.

B. Deployment Implications

For deployment, the most relevant decision-quality figure is 0.42, not the headline MTTR: at these settings full executes an accepted mitigation in fewer than half of its runs. Together with the rise in manual interventions, this campaign gives no support for running this class of system unattended and fully autonomously against real infrastructure. The results instead favor the graduated trust ladder of Section IV as the safer production default: shadow, approval, and autonomous modes, with a durable kill switch and a per-target action cap. They also argue against treating the research benchmark's default of full autonomy as a deployment recommendation. For now, the trust core is an architectural response to a question that our measurements raise; we have not yet measured whether it works.

C. The Manual-Intervention Mechanism Is Measured, Not Explained

Section VI reports that full and recovery_only increase manual interventions relative to baseline, contrary to the base paper's claim. The measurement itself is direct: the metric counts rows in the telemetry.manual_interventions table, to which every configuration writes. The mechanism behind the increase is not established. Repeated escalation

across the ticks of one open fault may dominate, repeated denials may end in human hand-offs, or the two may interact. Distinguishing these explanations would require a per-tick trace of proposal outcomes over the lifetime of a fault, which the current telemetry does not retain. We therefore leave the mechanism open rather than infer it from the aggregate count, and readers should not regard it as resolved in the system's favor.

D. Limitations and Threats to Validity

1) Threats to Validity: Eight threats apply directly. Simulated human. Every comparison against baseline inherits the log-normal on-call model; Section VI-G reports the break-even point rather than defending the model itself. Modelled baselines. rule_based and autoscale resolve covered faults after fixed, assumed latencies (Section V), so their advantage over the agents reflects those assumptions and is not a measurement of a deployed rule engine or autoscaler. Synthetic workload. The data come from a downscaled synthetic table with injected faults rather than from production traces, and the results may not carry over to real workloads. One live model family. The live arm used one reasoning model and one faster model from a single provider at temperature zero, so the decision-quality findings (Section VI) are properties of that pairing and may not generalize. Unequal replicate counts. baseline, rule_based, autoscale, and full each have 80 runs, whereas the four single-agent ablations have half as many. This cost trade-off was made before the campaign (Section V) and widens the ablations' confidence intervals relative to the headline comparisons. Nominal seed matching. Because the per-run seed depends on the configuration name (Section V), no two configurations share fault or human-latency randomness at a matched (scenario, replicate) index. We therefore used the unpaired rank-sum test as the primary test, and the retained signed-rank statistic should not be read as evidence from a paired design. Detection-clock asymmetry. Non-agent configurations detect at injection by construction, whereas agent configurations start the MTTR clock only when the monitoring agent flags the fault (Section III). The agents' recovery times therefore omit a detection interval that the non-agent figures include, which favors the agents in every MTTR comparison. Cost measurement. The metered term of Eq. (4) is integrated between two resource samples per run, which spans the 300 s control loop for agent configurations but only a few seconds for non-agent ones. In addition, the Airflow pool size and the streaming-worker target carry over from one run to the next because the campaign never reset them. Cost comparisons therefore rest mainly on the provisioning term, an assumption whose break-even point Section VI-G reports, and the metered part should not be read as a like-for-like measurement.

2) Scope of the Testbed: The testbed is a single machine. The resource-contention scenario applies real CPU stress, which is why runs cannot be parallelized and why the size of the campaign is bounded. The policy engine is pinned to one release, and lesson L6 (Section VII) shows that a newer release from the same vendor would have changed the adversarial result had the version not been pinned. We disclose this as a property of our evaluation, not as a defect we investigated in the policy engine. Only four fault types were injected, whereas a real pipeline's incident taxonomy is larger, and lesson L4 (Section VII) shows that mapping an injected fault type onto a real incident type is not automatic.

3) Evidence Gaps: Two components described in Section IV are implemented but not evaluated, and a third gap lies within a contribution that is otherwise evaluated. Cross-model generalization. The harness that could test the base paper's claim of model independence is unit-tested, but its live sweep against real provider models has not been run, so nothing here is evidence for or against that claim. The trust core. Graduated autonomy, the kill switch, and the blast-radius cap are implemented and unit-tested, but every configuration ran in one fixed mode, so no mode transition or kill-switch-to-halt latency was measured. Cost. The cost model computes a compute and storage split internally, but the committed per-run data keep only the combined total, and the metered term is confounded as described above.

Each gap has a concrete and bounded remedy. The cross-model sweep should be run against at least two live providers on the same scenario set. A dedicated drill should engage the kill switch and a mode transition under load and record the timing. For cost, the harness should reset the pool size and the worker target before each run, sample resources at a fixed cadence over a window that is identical for every configuration, and persist the ledger's component breakdown. The first two gaps are evidence this campaign did not collect; the third is a measurement flaw that a re-run with these changes would remove.

4) What Would Change Our Conclusion: A cross-model repetition of Section VI-E that reproduced the same order-of-magnitude mock-versus-live gap with a different provider would considerably strengthen the central claim. A repetition that did not would suggest that the gap is specific to this model pairing rather than a property of mock evaluation as a method, and that result would deserve an equally plain report.

IX. CONCLUSION AND FUTURE WORK

We implemented and evaluated a policy-gated multi-agent design for cloud pipeline governance using a live remote language model, explicit non-agent baselines, an adversarial evaluation against an independent oracle, and closed-form sensitivity analyses. The design holds up only in part. Against static orchestration it reduces recovery time by less than the original report claimed. It reduces cost by more, but almost entirely through an assumed provisioning gap. It increases manual interventions, and two idealized non-LLM comparators recover faster still. The most consequential result, however, concerns the evaluation itself. Run through the same code path, the same configuration supports very different conclusions under a

deterministic mock and under a live model: recovery time differs by roughly three orders of magnitude, and the mock also reports a perfect correct-mitigation score that the live model does not approach. For this class of system, which evaluation path produced a number matters at least as much as the number itself. The practical conclusion is deliberately narrow. It is not a verdict on LLM agents in operations generally, but a reason to ask how a given governance claim was evaluated, and to treat mock and live numbers as separate findings until both exist.

Future work falls into two parts. The first is the evidence this campaign did not collect: a live cross-model sweep to test the claim of model independence, an exercise of the trust core's mode transitions and kill-switch latency under load, a cost re-run with per-run state reset and a fixed measurement window, and the replacement of the two modelled baselines with deployed controllers and real production traces (Section VIII). The second is design work: replacing the static agent-priority bid with an explicit negotiation protocol, enabling the bounded-adaptation mechanism that is currently disabled to preserve determinism, and extending the evaluation to multi-cloud deployments and to formal verification of the policy layer.

DATA AND CODE AVAILABILITY

The implementation, the Rego policies, the per-run campaign data for all 560 runs, the adversarial-corpus results, and the scripts that regenerate every table, figure, and reported number in this paper are available at https://github.com/bodapatisaikrishna/agentic-cloud-pipeline-governance.

ACKNOWLEDGMENT

Generative AI tools (Claude, Anthropic) were used to assist with software development, analysis scripts, and editing of the manuscript text. The authors reviewed all AI-assisted output, checked every reported result against the committed campaign data, and take full responsibility for the content of this paper.

References

[1] H. Foidl, V. Golendukhina, R. Ramler, and M. Felderer, "Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers," J. Syst. Softw., vol. 207, p. 111855, 2024, doi: 10.1016/j.jss.2023.111855.

[2] Y. Chen et al., "Automatic root cause analysis via large language models for cloud incidents," in Proc. 19th Eur. Conf. Comput. Syst. (EuroSys), 2024, pp. 674–688, doi: 10.1145/3627703.3629553.

[3] Z. Wang et al., "RCAgent: Cloud root cause analysis by autonomous agents with tool-augmented large language models," in Proc. 33rd ACM Int. Conf. Inf. Knowl. Manage. (CIKM), 2024, pp. 4966–4974, doi: 10.1145/3627673.3680016.

[4] Y. Luo et al., "OpsAgent: An evolving multi-agent system for incident management in microservices," arXiv:2510.24145, 2025, doi: 10.48550/arXiv.2510.24145.

[5] A. M. Kirubakaran et al., "Governing cloud data pipelines with agentic AI," arXiv:2512.23737, 2025, doi: 10.48550/arXiv.2512.23737.

[6] X. Liu, T. Chen, W. Li, X. Hu, and H. Wei, "The sim-to-real gap of foundation model agents: A unified MDP perspective," arXiv:2606.07017, 2026, KDD 2026 Blue Sky Ideas Track, doi: 10.48550/arXiv.2606.07017.

[7] M. Zhu et al., "RealUserSim: Bridging the reality gap in agent benchmarking via grounded user simulation," arXiv:2605.20204, 2026, doi: 10.48550/arXiv.2605.20204.

[8] A. Gupta, "ReliabilityBench: Evaluating LLM agent reliability under production-like stress conditions," arXiv:2601.06112, 2026, doi: 10.48550/arXiv.2601.06112.

[9] K. Rzadca et al., "Autopilot: Workload autoscaling at Google," in Proc. 15th Eur. Conf. Comput. Syst. (EuroSys), 2020, doi: 10.1145/3342195.3387524.

[10] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track, 2024, doi: 10.48550/arXiv.2406.13352.

[11] H. Zhang et al., "Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in Proc. 13th Int. Conf. Learn. Represent. (ICLR), 2025, doi: 10.48550/arXiv.2410.02644.

[12] A. Joshi, T. Finin, K. P. Joshi, and L. Kagal, "Deontic policies for runtime governance of agentic AI systems," arXiv:2606.19464, 2026, doi: 10.48550/arXiv.2606.19464.

[13] A. C. Sanna, "Cross-LLM generalization of behavioral backdoor detection in AI agent supply chains," arXiv:2511.19874, 2025, doi: 10.48550/arXiv.2511.19874.

[14] L. Zhang, Y.-Y. Chu, and R. Krishnan, "Toward agentic governance: What shapes LLM-agent intervention in public forums?" arXiv:2606.00603, 2026, doi: 10.48550/arXiv.2606.00603.

[15] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proc. 11th Int. Conf. Learn. Represent. (ICLR), 2023, doi: 10.48550/arXiv.2210.03629.

[16] F. F. Xu et al., "TheAgentCompany: Benchmarking LLM agents on consequential real world tasks," arXiv:2412.14161, 2024, doi: 10.48550/arXiv.2412.14161.

[17] J. Rystrøm, C. Schmitz, K. Korgul, J. Batzner, and C. Russell, "Agent benchmarks fail public sector requirements," arXiv:2601.20617, 2026, doi: 10.48550/arXiv.2601.20617.

[18] D. Hadfield-Menell, A. Dragan, P. Abbeel, and S. Russell, "The off-switch game," in Proc. 26th Int. Joint Conf. Artif. Intell. (IJCAI), 2017, pp. 220–227, doi: 10.24963/ijcai.2017/32.

[19] H. Zheng, Q. Dong, R. K. Depena, J. D. Bhatia, F. Xiao, and P. Xu, "Separating capability from permission: A governance framework for agentic AI autonomy levels," arXiv:2607.23438, 2026, doi: 10.48550/arXiv.2607.23438.

[20] Cloud Native Computing Foundation, "Open Policy Agent: policy-based control for cloud native environments," https://www.openpolicyagent.org, 2016, accessed 2026-09-21.

[21] K. Zhang, Y. Peng, K. Jiang, and Y. Zhang, "ActGov: Governing LLM agent actions via policy-constrained validation," arXiv:2609.24446, 2026, doi: 10.48550/arXiv.2609.24446.

[22] Y. Chen et al., "AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds," arXiv:2501.06706, 2025, doi: 10.48550/arXiv.2501.06706.

[23] J. Barnes, "OpenSec: Measuring incident response agent calibration under adversarial evidence," arXiv:2601.21083, 2026, doi: 10.48550/arXiv.2601.21083.

[24] A. Basiri et al., "Chaos engineering," IEEE Software, vol. 33, no. 3, pp. 35–41, 2016, doi: 10.1109/MS.2016.60.

[25] B. Efron, "Bootstrap methods: Another look at the jackknife," Ann. Statist., vol. 7, no. 1, 1979, doi: 10.1214/aos/1176344552.

[26] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," Ann. Math. Statist., vol. 18, no. 1, pp. 50–60, 1947, doi: 10.1214/aoms/1177730491.

[27] F. Wilcoxon, "Individual comparisons by ranking methods," Biometrics Bull., vol. 1, no. 6, p. 80, 1945, doi: 10.2307/3001968.

[28] S. Holm, "A simple sequentially rejective multiple test procedure," Scand. J. Statist., vol. 6, no. 2, pp. 65–70, 1979.

[29] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," Psychol. Bull., vol. 114, no. 3, pp. 494–509, 1993, doi: 10.1037/0033-2909.114.3.494.

[30] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” J. Amer. Statist. Assoc., vol. 22, no. 158, pp. 209–212, 1927, doi: 10.1080/01621459.1927.10502953.